CSSE 220 – Object-Oriented Software Development
Rose-Hulman Institute of Technology

Worksheet 12: Abstract Classes + Scanner

Name (Print):_____ Section:_____

1. **Complete the abstract class.** Fill in the two missing keywords.

```
1 public ________ class GameEntity {
2     protected int x, y;
3     public void move(int dx, int dy) {
4         x += dx;
5         y += dy;
6     }
7     public ________ void update();
8 }
```

2. **Rules of abstract classes.** Circle *all* true statements.

   (a) You can instantiate an abstract class with `new GameEntity()`.

   (b) An abstract method has no body.

   (c) An abstract class may have both abstract and concrete methods.

   (d) An abstract class may declare constructors.

   (e) An abstract class may have instance fields.

   (f) A class that extends an abstract class must always override every method.

3. **What compiles?** Assume `GameEntity` is the abstract class. Zombie extends GameEntity.

```
1 GameEntity a = new GameEntity(); // A
2 GameEntity b = new Zombie(); // B
3 Zombie z = new Zombie(); // C
```

4. **Design choices.** Choose *regular classes only*, *concrete superclass*, *abstract class*, or *interface*. Give a short justification for two of your answers.

   (a) `Player`, `Zombie`, `Skeleton`: common position and `move()`, but each must implement its own `update()`. _____

   (b) `Circle`, `Square`, `Triangle`: each computes `area()`, but no common state or implementation is required. _____

   (c) `Car`, `Truck`, `Motorcycle`: share fields and fully implemented driving methods; none require special behavior. _____

   (d) `Bird`, `Airplane`, `Drone`: unrelated types, all can `fly()`. _____

   (e) `Book`, `Library`, `Student`: no shared implementation or common capability is specified. _____

5. **Key distinction.** Fill in the blanks.
   An abstract class is useful for shared _____ and required _____.
   An interface describes a shared _____.

   Can a class extend two classes? _____ Can it implement two interfaces? _____

6. **Scanner Complete the input code.** Write the import, constructor argument, and methods that read an integer and an entire line.

```
import java.util._____;

public class AdventureSetup {
    public static void main(String[] args) {
        Scanner input = new Scanner(_____);
        System.out.print("How many lives? ");
        int lives = input._____();
        input.nextLine();  // consume the rest of this line
        System.out.print("Name your world: ");
        String world = input._____();
        System.out.println(world + " has " + lives + " lives.");
    }
}
```

7. **Predict the input.** The user types `Ender Dragon` and presses Enter. What does each method return when called first on that fresh input?

   (a) `next()` → _____

   (b) `nextLine()` → _____

8. **Debug a Scanner surprise.** The player enters `5`, presses Enter, then types `Nether Fortress` and presses Enter.

```
Scanner input = new Scanner(System.in);
System.out.print("Lives: ");
int lives = input.nextInt();
System.out.print("World: ");
String world = input.nextLine();
System.out.println("World = [" + world + "]");
```

   What is printed on the last line? _____

   Fix: Add exactly one line in the correct location so the world name is read as intended.

9. **Input types and errors.** For each task, pick a Scanner method: `next()`, `nextLine()`, or `nextInt()`.

   (a) Read a player name that may contain spaces: _____

   (b) Read a quantity of diamonds as an integer: _____

   (c) Read a one-word command such as `attack`: _____

   What happens if the user types `three` when `nextInt()` expects an integer?